

🌐 ДОМЕН 6: СЕТИ И РАСПРЕДЕЛЕННЫЕ СИСТЕМЫ — ПОЛНАЯ ДЕТАЛИЗАЦИЯ (ОТ НУЛЯ ДО ЭКСПЕРТА)

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 6.1: ОСНОВЫ СЕТЕЙ (JUNIOR LEVEL)

6.1.1. Модель OSI и TCP/IP (Junior Level)

№ 6.1.1.1 — Модель OSI

- **Что нужно знать:** 7 уровней (Физический, Канальный, Сетевой, Транспортный, Сеансовый, Представления, Прикладной)
- **Вопросы на собеседовании:** "Какие уровни есть в модели OSI?"
- **Практика:** Нарисовать схему OSI и объяснить, что делает каждый уровень

№ 6.1.1.2 — TCP/IP модель

- **Что нужно знать:** 4 уровня (Сетевой интерфейс, Интернет, Транспортный, Прикладной)
- **Вопросы на собеседовании:** "В чем разница между OSI и TCP/IP?"
- **Практика:** Сравнить OSI и TCP/IP модели

№ 6.1.1.3 — IP-адресация

- **Что нужно знать:** IPv4 (32 бита, 4 октета), IPv6 (128 бит), маска подсети, CIDR
- **Вопросы на собеседовании:** "Что такое маска подсети и зачем она нужна?"
- **Практика:** Вычислить адрес сети для IP 192.168.1.100/24

№ 6.1.1.4 — Порты

- **Что нужно знать:** well-known (0-1023), registered (1024-49151), ephemeral (49152-65535)

- **Вопросы на собеседовании:** "Какие порты используются для HTTP, HTTPS, SSH, DNS?"
- **Практика:** Написать программу, слушающую порт 8080

№ 6.1.1.5 — TCP

- **Что нужно знать:** трехстороннее рукопожатие (SYN, SYN-ACK, ACK), сегментация, управление потоком
- **Вопросы на собеседовании:** "Как устанавливается TCP-соединение?"
- **Практика:** Запустить Wireshark и показать TCP handshake

№ 6.1.1.6 — UDP

- **Что нужно знать:** без установления соединения, без гарантии доставки, быстрый
- **Вопросы на собеседовании:** "В чем разница между TCP и UDP?"
- **Практика:** Написать UDP-эхо-сервер на C++ и сравнить с TCP

№ 6.1.1.7 — DNS

- **Что нужно знать:** разрешение доменных имен, рекурсивный/итеративный запрос
- **Вопросы на собеседовании:** "Как работает DNS?"
- **Практика:** Выполнить nslookup google.com и показать, как он работает

№ 6.1.1.8 — DHCP

- **Что нужно знать:** автоматическая настройка IP, ARP (Address Resolution Protocol)
- **Вопросы на собеседовании:** "Как устройство получает IP-адрес?"
- **Практика:** Проанализировать DHCP-запросы в Wireshark

6.1.2. HTTP/HTTPS (Junior → Middle Level)

№ 6.1.2.1 — HTTP

- **Что нужно знать:** структура запроса (метод, URL, заголовки, тело), структура ответа (статус, заголовки, тело)
- **Вопросы на собеседовании:** "Как выглядит HTTP-запрос и ответ?"
- **Практика:** Написать HTTP-клиент на C++ (без библиотек)

№ 6.1.2.2 — Методы

- **Что нужно знать:** GET (получение), POST (создание), PUT (замена), PATCH (частичное обновление), DELETE (удаление)
- **Вопросы на собеседовании:** "В чем разница между PUT и PATCH?"
- **Практика:** Создать REST API с этими методами

№ 6.1.2.3 — Статусы

- **Что нужно знать:** 2xx (успех), 3xx (редиректы), 4xx (ошибки клиента), 5xx (ошибки сервера)
- **Вопросы на собеседовании:** "Что означает статус 404? 500? 302?"
- **Практика:** Написать обработку статусов в клиенте

№ 6.1.2.4 — Заголовки

- **Что нужно знать:** Content-Type, Authorization, Accept, User-Agent, Cookie, Cache-Control
- **Вопросы на собеседовании:** "Какие заголовки важны для REST API?"
- **Практика:** Настроить CORS и авторизацию в [ASP.NET](#) Core

№ 6.1.2.5 — HTTPS

- **Что нужно знать:** TLS/SSL, сертификаты, шифрование, асимметричное/симметричное шифрование
- **Вопросы на собеседовании:** "Как работает HTTPS?"
- **Практика:** Сгенерировать самоподписанный сертификат и запустить HTTPS-сервер

№ 6.1.2.6 — HTTP/2

- **Что нужно знать:** мультиплексирование, сжатие заголовков, server push
- **Вопросы на собеседовании:** "В чем преимущество HTTP/2 перед HTTP/1.1?"
- **Практика:** Включить HTTP/2 в [ASP.NET](#) Core и сравнить производительность

№ 6.1.2.7 — HTTP/3 (QUIC)

- **Что нужно знать:** основан на UDP, уменьшение задержки
- **Вопросы на собеседовании:** "Что такое QUIC и зачем он нужен?"

- **Практика:** Изучить QUIC и его преимущества для мобильных устройств

№ 6.1.2.8 — WebSockets

- **Что нужно знать:** двусторонняя связь в реальном времени
 - **Вопросы на собеседовании:** "Когда использовать WebSockets вместо HTTP?"
 - **Практика:** Написать чат с WebSockets на C#
-

6.1.3. Практические задания (Junior → Middle Level)

№ 6.1.3.1

- **Задание:** Написать TCP-эхо-сервер на C++ и клиент на C#
- **Что проверяет:** Понимание TCP

№ 6.1.3.2

- **Задание:** Создать REST API на [ASP.NET](#) Core с методами CRUD
- **Что проверяет:** Знание HTTP

№ 6.1.3.3

- **Задание:** Написать HTTP-клиент на C++ для GET-запроса
- **Что проверяет:** Понимание HTTP

№ 6.1.3.4

- **Задание:** Проанализировать трафик в Wireshark и найти HTTP-запросы
 - **Что проверяет:** Навыки анализа сетей
-

РАЗДЕЛ 6.2: API И RPC (JUNIOR → SENIOR)

6.2.1. REST и gRPC (Middle Level)

№ 6.2.1.1 — REST

- **Что нужно знать:** принципы (клиент-сервер, отсутствие состояния, кешируемость, единообразие интерфейса)
- **Вопросы на собеседовании:** "Что такое RESTful API?"
- **Практика:** Спроектировать REST API для управления пользователями

№ 6.2.1.2 — OpenAPI / Swagger

- **Что нужно знать:** документирование API, генерация клиентов
- **Вопросы на собеседовании:** "Как документировать REST API?"
- **Практика:** Добавить Swagger в [ASP.NET](#) Core

№ 6.2.1.3 — gRPC

- **Что нужно знать:** Protocol Buffers (определение сообщений и сервисов), HTTP/2
- **Вопросы на собеседовании:** "В чем преимущество gRPC перед REST?"
- **Практика:** Создать gRPC-сервис на C# и клиент на C++

№ 6.2.1.4 — Protocol Buffers

- **Что нужно знать:** синтаксис .proto, типы (string, int32, repeated), optional
- **Вопросы на собеседовании:** "Как определить gRPC-сервис?"
- **Практика:** Написать .proto для сервиса UserService

№ 6.2.1.5 — gRPC (4 типа)

- **Что нужно знать:** Unary (один запрос-ответ), Server Streaming, Client Streaming, Bidirectional Streaming
- **Вопросы на собеседовании:** "Когда использовать Server Streaming?"
- **Практика:** Реализовать server streaming для отправки большого набора данных

№ 6.2.1.6 — GraphQL

- **Что нужно знать:** гибкие запросы, выбор полей, алмазы, подписки
- **Вопросы на собеседовании:** "Когда использовать GraphQL вместо REST?"
- **Практика:** Написать GraphQL-сервер на C# (Hot Chocolate)

№ 6.2.1.7 — Versioning API

- **Что нужно знать:** URL (/api/v1/users), заголовки (Accept-Version)
 - **Вопросы на собеседовании:** "Как версионировать API?"
 - **Практика:** Добавить версионирование в [ASP.NET](#) Core
-

6.2.2. Практические задания (Middle Level)

№ 6.2.2.1

- **Задание:** Создать gRPC-сервис для получения пользователей (Unary и Server Streaming)
- **Что проверяет:** Знание gRPC

№ 6.2.2.2

- **Задание:** Создать REST API с документацией Swagger
- **Что проверяет:** Знание REST

№ 6.2.2.3

- **Задание:** Создать GraphQL-сервер с запросами и мутациями
- **Что проверяет:** Знание GraphQL

№ 6.2.2.4

- **Задание:** Написать клиент на C++ для gRPC-сервиса
- **Что проверяет:** Кросс-языковая коммуникация

РАЗДЕЛ 6.3: МИКРОСЕРВИСЫ И ОЧЕРЕДИ (MIDDLE → SENIOR)

6.3.1. Микросервисная архитектура (Middle Level)

№ 6.3.1.1 — Монолит vs Микросервисы

- **Что нужно знать:** преимущества, недостатки
- **Вопросы на собеседовании:** "Когда использовать монолит, а когда микросервисы?"
- **Практика:** Спроектировать монолит и декомпозировать на микросервисы

№ 6.3.1.2 — Синхронная vs Асинхронная коммуникация

- **Что нужно знать:** REST/gRPC (синхронная) vs очереди (асинхронная)
- **Вопросы на собеседовании:** "В чем разница между синхронной и асинхронной коммуникацией?"
- **Практика:** Спроектировать систему с асинхронной коммуникацией

№ 6.3.1.3 — Service Discovery

- **Что нужно знать:** Consul, etcd, ZooKeeper (регистрация сервисов)

- **Вопросы на собеседовании:** "Как сервисы находят друг друга?"
- **Практика:** Настроить Consul для регистрации сервисов

№ 6.3.1.4 — API Gateway

- **Что нужно знать:** маршрутизация, авторизация, балансировка (Kong, Ocelot)
- **Вопросы на собеседовании:** "Зачем нужен API Gateway?"
- **Практика:** Настроить Ocelot для маршрутизации в [ASP.NET](#) Core

№ 6.3.1.5 — Circuit Breaker

- **Что нужно знать:** предотвращение каскадных отказов (Polly, Hystrix)
- **Вопросы на собеседовании:** "Что такое Circuit Breaker и как он работает?"
- **Практика:** Реализовать Circuit Breaker с Polly

№ 6.3.1.6 — Retry и Timeout

- **Что нужно знать:** повторные попытки, таймауты
- **Вопросы на собеседовании:** "Как обрабатывать временные сбои?"
- **Практика:** Настроить Retry и Timeout с Polly

6.3.2. Message Queues (Senior Level)

№ 6.3.2.1 — RabbitMQ

- **Что нужно знать:** exchanges (direct, topic, fanout), queues, routing keys
- **Вопросы на собеседовании:** "Как работает RabbitMQ?"
- **Практика:** Поднять RabbitMQ, написать producer и consumer на C#

№ 6.3.2.2 — Apache Kafka

- **Что нужно знать:** топики, партиции, оффсеты, consumer groups
- **Вопросы на собеседовании:** "В чем отличие Kafka от RabbitMQ?"
- **Практика:** Поднять Kafka, написать producer и consumer на C#

№ 6.3.2.3 — Kafka (гарантии)

- **Что нужно знать:** At-most-once, At-least-once, Exactly-once
- **Вопросы на собеседовании:** "Как Kafka обеспечивает Exactly-Once?"

- **Практика:** Настроить Exactly-Once для Kafka producer

№ 6.3.2.4 — Dead Letter Queue

- **Что нужно знать:** обработка "битых" сообщений
- **Вопросы на собеседовании:** "Что такое Dead Letter Queue?"
- **Практика:** Настроить DLQ в RabbitMQ

№ 6.3.2.5 — MassTransit

- **Что нужно знать:** абстракция над очередями (RabbitMQ, Azure Service Bus)
- **Вопросы на собеседовании:** "Как использовать MassTransit?"
- **Практика:** Написать MassTransit producer/consumer для RabbitMQ

№ 6.3.2.6 — Kafka Streams

- **Что нужно знать:** обработка потоков, stateful/stateless операции
- **Вопросы на собеседовании:** "Как обрабатывать данные в реальном времени?"
- **Практика:** Написать Kafka Streams приложение на C#

6.3.3. Практические задания (Middle → Senior Level)

№ 6.3.3.1

- **Задание:** Спроектировать микросервисную архитектуру для интернет-магазина
- **Что проверяет:** Знание архитектуры

№ 6.3.3.2

- **Задание:** Настроить RabbitMQ для обработки заказов (order created → payment → inventory)
- **Что проверяет:** Знание очередей

№ 6.3.3.3

- **Задание:** Написать Kafka producer/consumer для логирования действий пользователей
- **Что проверяет:** Знание Kafka

№ 6.3.3.4

- **Задание:** Реализовать Circuit Breaker с Polly для HTTP-запросов
 - **Что проверяет:** Знание устойчивости
-

РАЗДЕЛ 6.4: РАСПРЕДЕЛЕННЫЕ СИСТЕМЫ (SENIOR → STAFF)

6.4.1. Консенсус (Senior Level)

№ 6.4.1.1 — 2PC (Two-Phase Commit)

- **Что нужно знать:** подготовка, фиксация, проблемы (блокировка, single point of failure)
- **Вопросы на собеседовании:** "Что такое 2PC и в чем его недостатки?"
- **Практика:** Смоделировать 2PC для двух сервисов

№ 6.4.1.2 — Saga Pattern

- **Что нужно знать:** оркестрация (центральный координатор) vs хореография (обмен событиями)
- **Вопросы на собеседовании:** "В чем разница между оркестрацией и хореографией?"
- **Практика:** Реализовать Saga для заказа (Order, Payment, Inventory)

№ 6.4.1.3 — Paxos

- **Что нужно знать:** алгоритм консенсуса, роли (proposer, acceptor, learner)
- **Вопросы на собеседовании:** "Как работает алгоритм Paxos?"
- **Практика:** Изучить Paxos и написать конспект

№ 6.4.1.4 — Raft

- **Что нужно знать:** выбор лидера, репликация логов, безопасность
- **Вопросы на собеседовании:** "Чем Raft отличается от Paxos?"
- **Практика:** Реализовать упрощенный Raft на C++ (Follower, Candidate, Leader)

№ 6.4.1.5 — ZooKeeper / etcd

- **Что нужно знать:** координация, обнаружение сервисов, выборы лидера
- **Вопросы на собеседовании:** "Как использовать etcd для координации?"

- **Практика:** Развернуть etcd и использовать для Service Discovery

№ 6.4.1.6 — Distributed Consensus в БД

- **Что нужно знать:** как работают distributed transactions в Spanner, CockroachDB
 - **Вопросы на собеседовании:** "Как Google Spanner обеспечивает глобальную согласованность?"
 - **Практика:** Изучить TrueTime в Spanner
-

6.4.2. Согласованность и CAP (Staff Level)

№ 6.4.2.1 — CAP-теорема

- **Что нужно знать:** Consistency, Availability, Partition Tolerance (можно выбрать только 2)
- **Вопросы на собеседовании:** "Что такое CAP-теорема?"
- **Практика:** Выбрать БД для системы с требованием высокой доступности

№ 6.4.2.2 — PACELC

- **Что нужно знать:** расширение CAP (Partition, Availability, Consistency, Latency)
- **Вопросы на собеседовании:** "В чем отличие PACELC от CAP?"
- **Практика:** Изучить PACELC для Cassandra, DynamoDB

№ 6.4.2.3 — Линейность (Linearizability)

- **Что нужно знать:** строгая согласованность (как один узел)
- **Вопросы на собеседовании:** "Что такое линейность?"
- **Практика:** Спроектировать систему с линейностью

№ 6.4.2.4 — Eventual Consistency

- **Что нужно знать:** согласованность в конечном итоге
- **Вопросы на собеседовании:** "Когда использовать Eventual Consistency?"
- **Практика:** Спроектировать систему с Eventually Consistent кешем

№ 6.4.2.5 — Strong Consistency vs Eventual Consistency

- **Что нужно знать:** trade-offs
- **Вопросы на собеседовании:** "Что выбрать: strong или eventual consistency?"
- **Практика:** Сравнить производительность двух подходов

№ 6.4.2.6 — Vector Clocks

- **Что нужно знать:** определение порядка событий в распределенных системах
- **Вопросы на собеседовании:** "Как Vector Clocks работают?"
- **Практика:** Реализовать Vector Clocks для двух узлов

№ 6.4.2.7 — Lamport Timestamps

- **Что нужно знать:** логические часы, упорядочивание событий
- **Вопросы на собеседовании:** "В чем отличие Lamport Timestamps от Vector Clocks?"
- **Практика:** Реализовать Lamport Timestamps

№ 6.4.2.8 — Google TrueTime

- **Что нужно знать:** GPS + атомные часы для глобальной согласованности
- **Вопросы на собеседовании:** "Как TrueTime работает в Spanner?"
- **Практика:** Изучить TrueTime и его преимущества

6.4.3. Практические задания (Senior → Staff Level)

№ 6.4.3.1

- **Задание:** Реализовать Saga для распределенной транзакции (Order → Payment → Shipping)
- **Что проверяет:** Понимание Saga

№ 6.4.3.2

- **Задание:** Реализовать упрощенную модель Raft на C++ (состояния Follower, Candidate, Leader)
- **Что проверяет:** Понимание консенсуса

№ 6.4.3.3

- **Задание:** Спроектировать систему с Vector Clocks для разрешения конфликтов
- **Что проверяет:** Понимание согласованности

№ 6.4.3.4

- **Задание:** Сравнить performance БД с Strong Consistency и Eventual Consistency
- **Что проверяет:** Понимание trade-offs

РАЗДЕЛ 6.5: СТРИМИНГОВЫЕ СИСТЕМЫ (SENIOR → STAFF)

6.5.1. Kafka Streams и Flink (Senior Level)

№ 6.5.1.1 — Kafka Streams

- **Что нужно знать:** обработка потоков в реальном времени, DSL
- **Вопросы на собеседовании:** "Что такое Kafka Streams?"
- **Практика:** Написать Kafka Streams приложение для подсчета слов

№ 6.5.1.2 — Stateful vs Stateless

- **Что нужно знать:** операции без состояния (map, filter) и с состоянием (aggregate, join)
- **Вопросы на собеседовании:** "В чем разница между stateful и stateless операциями?"
- **Практика:** Написать stateful операцию для скользящего среднего

№ 6.5.1.3 — Watermarks

- **Что нужно знать:** управление временем в стримах (event time vs processing time)
- **Вопросы на собеседовании:** "Что такое Watermark и зачем он нужен?"
- **Практика:** Настроить Watermark для обработки задержанных событий

№ 6.5.1.4 — Exactly-Once

- **Что нужно знать:** как Kafka Streams обеспечивает Exactly-Once (Idempotence, Transactions)
- **Вопросы на собеседовании:** "Как реализовать Exactly-Once в Kafka Streams?"
- **Практика:** Настроить Exactly-Once для Kafka Streams приложения

№ 6.5.1.5 — Apache Flink

- **Что нужно знать:** более мощная стриминговая платформа
- **Вопросы на собеседовании:** "В чем отличие Flink от Kafka Streams?"
- **Практика:** Написать Flink-приложение на Java для оконной обработки

№ 6.5.1.6 — Windowing

- **Что нужно знать:** Tumbling, Sliding, Session windows
- **Вопросы на собеседовании:** "Какие типы окон есть в стриминговых системах?"
- **Практика:** Реализовать Tumbling Window для агрегации

№ 6.5.1.7 — Checkpointing

- **Что нужно знать:** сохранение состояния для восстановления
- **Вопросы на собеседовании:** "Как Flink обеспечивает отказоустойчивость?"
- **Практика:** Настроить Checkpointing в Flink

6.5.2. Практические задания (Senior Level)

№ 6.5.2.1

- **Задание:** Написать Kafka Streams приложение для подсчета событий по типу за 5 минут
- **Что проверяет:** Понимание стриминга

№ 6.5.2.2

- **Задание:** Написать приложение для скользящего среднего за 10 секунд на Kafka Streams
- **Что проверяет:** Понимание окон

№ 6.5.2.3

- **Задание:** Написать Flink-приложение для анализа логов в реальном времени
- **Что проверяет:** Знание Flink

№ 6.5.2.4

- **Задание:** Настроить Exactly-Once для Kafka producer

- **Что проверяет:** Знание гарантий доставки
-

РАЗДЕЛ 6.6: РАСПРЕДЕЛЕННЫЕ ПАТТЕРНЫ (STAFF LEVEL)

6.6.1. CRDT и распределенные структуры (Staff Level)

№ 6.6.1.1 — CRDT (Conflict-free Replicated Data Types)

- **Что нужно знать:** для согласованности без координации
- **Вопросы на собеседовании:** "Что такое CRDT?"
- **Практика:** Реализовать CRDT-счетчик (G-Counter)

№ 6.6.1.2 — Distributed Logging

- **Что нужно знать:** ELK Stack (Elasticsearch, Logstash, Kibana), OpenSearch
- **Вопросы на собеседовании:** "Как организовать централизованное логирование?"
- **Практика:** Настроить ELK для логирования микросервисов

№ 6.6.1.3 — Distributed Tracing

- **Что нужно знать:** OpenTelemetry, Jaeger, Zipkin
- **Вопросы на собеседовании:** "Как отследить запрос в распределенной системе?"
- **Практика:** Добавить OpenTelemetry в [ASP.NET](#) Core и увидеть трейсы в Jaeger

№ 6.6.1.4 — Service Mesh

- **Что нужно знать:** Istio, Linkerd (управление трафиком, безопасность, наблюдаемость)
- **Вопросы на собеседовании:** "Что такое Service Mesh и зачем он нужен?"
- **Практика:** Установить Istio и настроить маршрутизацию

№ 6.6.1.5 — Chaos Engineering

- **Что нужно знать:** тестирование устойчивости с помощью хаос-монстров
- **Вопросы на собеседовании:** "Что такое Chaos Engineering?"

- **Практика:** Использовать Chaos Mesh для проверки отказоустойчивости

№ 6.6.1.6 — Distributed Cache

- **Что нужно знать:** Redis Cluster, Hazelcast
 - **Вопросы на собеседовании:** "Как организовать распределенный кеш?"
 - **Практика:** Настроить Redis Cluster для кеширования
-

6.6.2. Практические задания (Staff Level)

№ 6.6.2.1

- **Задание:** Реализовать CRDT-счетчик для распределенной системы
- **Что проверяет:** Понимание CRDT

№ 6.6.2.2

- **Задание:** Настроить Jaeger для трейсинга в микросервисах
- **Что проверяет:** Знание Observability

№ 6.6.2.3

- **Задание:** Настроить ELK для централизованного логирования
- **Что проверяет:** Знание логирования

№ 6.6.2.4

- **Задание:** Настроить Redis Cluster и протестировать отказоустойчивость
 - **Что проверяет:** Знание распределенных кешей
-

🔗 СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

OSI модель, TCP vs UDP

HTTP методы и статусы

REST API: принципы

DNS, IP-адресация, порты

Базовое понимание очередей (RabbitMQ)

Middle Level (должен знать 100%)

HTTP/2, HTTP/3, WebSockets

gRPC и Protocol Buffers

OpenAPI / Swagger

Микросервисы: синхронная vs асинхронная коммуникация

RabbitMQ: exchanges, queues, routing

Kafka: топики, партиции, оффсеты

Senior Level (должен знать 100%)

Circuit Breaker, Retry, Timeout

Service Discovery (Consul, etcd)

API Gateway

Kafka: consumer groups, Exactly-Once

Kafka Streams: stateful/stateless, windows

Saga Pattern: оркестрация vs хореография

2PC, распределенные транзакции

CAP-теорема, PACELC

Staff Level (должен знать 100%)

Raft, Paxos (консенсус)

Vector Clocks, Lamport Timestamps

Google TrueTime

CRDT (Conflict-free Replicated Data Types)

Service Mesh (Istio, Linkerd)

Distributed Tracing (OpenTelemetry, Jaeger)

Chaos Engineering

Распределенные транзакции (Saga, TCC)

🦾 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 6

Задание: Разработать распределенную систему обработки заказов интернет-магазина.

Шаги:

1. **Сервисы:** Order Service, Payment Service, Inventory Service, Notification Service (микросервисы)
2. **Коммуникация:** синхронная (gRPC) для критических операций, асинхронная (Kafka) для событий
3. **Saga:** реализовать Saga (хореографию) для заказа (Order → Payment → Inventory → Notification)
4. **Устойчивость:** добавить Circuit Breaker, Retry, Timeout
5. **Наблюдаемость:** OpenTelemetry + Jaeger для трейсов, Prometheus для метрик
6. **Координация:** etcd для Service Discovery
7. **Кеш:** Redis для кеширования популярных товаров
8. **Тестирование:** написать тесты на отказоустойчивость (Chaos Engineering)

Проверка:

- Система должна обрабатывать 1000 заказов/сек
- При падении одного сервиса, остальные должны продолжать работу
- Все заказы должны быть обработаны Exactly-Once
- Время отклика на заказ < 500 мс